ap-forecast-sample · docs/ap_forecast.md

Code documentation by the Data Science agent · FEAT-ZD-48600

ap_forecast module reference

Package version `0.10.0`. Python 3.9, standard library only. All functions are pure and in-memory except `load_config`, which reads `models.json` next to `explain.py`. Nothing in the package sends a request to the LLM provider: `build_explanation_request` only builds it.

Module	Public names	Purpose
`predict.py`	`predict_delay_days`	Baseline predicted payment delay in days from a vendor's history
`risk.py`	`vendor_risk`, `HIGH_ABOVE_DAYS`, `MEDIUM_ABOVE_DAYS`	Rule-based risk level (`'high'`, `'medium'`, `'low'`); no LLM, no config read (STORY-03)
`explain.py`	`build_explanation_request`, `load_config`, `CONFIG`	LLM request that explains a prediction; optional risk level in the prompt (STORY-04)
`models.json`	—	Provider, model and `max_tokens` for the explanation request

`ap_forecast.__all__` is `["predict_delay_days", "build_explanation_request", "vendor_risk"]`.

predict_delay_days

```
predict_delay_days(history: Iterable[int]) -> int
```

- **Input:** one vendor's past delays in days. Values are converted with `int()`; negative values (early payments) count as 0.
- **Output:** the rounded mean of the values, or `0` for an empty history.

```
>>> predict_delay_days([10, 20, 30])
20
>>> predict_delay_days([])
0
```

vendor_risk (new in 0.10.0)

```
vendor_risk(history: Iterable[int]) -> str
```

- **Input:** one vendor's delay history, in the same form as `predict_delay_days` takes (the value of one vendor in

`ap_pipeline`'s `delays_by_vendor` result). The history is read once, so a generator is accepted.

- **Output:** `'high'` when `predict_delay_days(history) > HIGH_ABOVE_DAYS` (30), `'medium'` when it is

`> MEDIUM_ABOVE_DAYS` (10), otherwise `'low'`. The comparison is strict and on the rounded prediction.

- **Edge cases:** an empty history predicts 0, so `'low'`; a history of early payments only predicts 0, so `'low'`.

A value that `int()` cannot convert raises from `predict_delay_days`, unchanged.

- **No LLM:** `risk.py` imports only `predict`; it never calls `load_config` or reads `models.json`.

```
>>> vendor_risk([31])
'high'
>>> vendor_risk([30])        # exactly 30 is not above 30
```

```
'medium'
>>> vendor_risk([10])        # exactly 10 is not above 10
'low'
>>> vendor_risk([])
'low'
```

Building the vendor-to-risk map used by `ap_aging.overdue_with_risk`:

```
risk_by_vendor = {vendor: vendor_risk(history) for vendor, history in delays.items()}
```

build_explanation_request (changed in 0.10.0: new optional `risk`)

```
build_explanation_request(vendor: str, predicted_days: int, config=None, risk=None) -> dict
```

- **Inputs:** `vendor` and `predicted_days` go into the prompt. `config` is a dict with `provider`, `model` and

`max_tokens`; when it is not given, `models.json` is read. `risk` is optional and is the last parameter, so every existing positional or keyword call works unchanged.

- **Output:** `{"provider", "model", "max_tokens", "messages": [{"role": "user", "content": prompt}]}`.
- **Risk:** when `risk` is not `None`, the prompt ends with `" Risk level: <risk>."`. When `risk` is missing or

`None`, the request is identical to the one built before 0.10.0. The value is not validated.

- **Data sent:** with `risk`, the risk label becomes part of the prompt that the caller sends to the configured

provider. Provider and model are unchanged.

```
>>> req = build_explanation_request("Synthetic Vendor A", 34, {"provider": "p", "model": "m", "max_tokens": 10}, risk="h
>>> req["messages"][0]["content"]
'Explain in one sentence why vendor Synthetic Vendor A is predicted to be paid 34 days late. Risk level: high.'
```

load_config

```
load_config() -> dict
```

Reads and returns `models.json` (`provider`, `model`, `max_tokens`).